

EKF SLAM for a Two-Wheeled Robot

Sebastian Ruiz

29 September 2023

1 Introduction

Simultaneous localization and mapping (SLAM) is the computational problem of constructing or updating a map of an unknown environment while simultaneously keeping track of an agent's location within it.

This document describes the Extended Kalman Filter (EKF) SLAM model implementation for a 2 wheeled robot. The landmarks that are being detected and the position of the robot are all modelled in 2D space.

Suppose the robot detects landmarks (e.g. aruco markers). For each landmark (with ID), a distance and angle to the landmark, relative to the robot, is estimated.

The EKF SLAM model allows the landmarks to be tracked relative to the robot. After the robot has navigated the space it is in for some time, it will have an accurate representation of where the landmarks are relative to itself.

Watch this video: EKF-SLAM (Cyrill Stachniss) <https://www.youtube.com/watch?v=X30sEgIws0g>
With accompanying slides: <http://ais.informatik.uni-freiburg.de/teaching/ws12/mapping/pdf/slam04-ekf-slam.pdf>

2 World Coordinate System

We represent the world as a 2D plane. The robot is at angle θ to the x -axis.

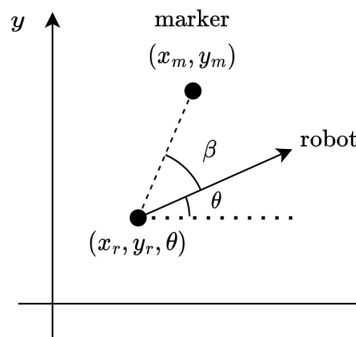


Figure 2.1: World coordinate system, with robot at position (x_r, y_r) with angle θ . A marker is shown with position (x_m, y_m) in world coordinates, with angle β to the robot.

3 Robot Motion Model

We model a two wheeled robot. The robot moves by turning the left and right wheels. The distance travelled by the left and right wheels determines whether the robot drives in a straight line, drives to the left or drives to the right.

In **Figure 3.1**, the red dot represents the robot and the arrow the robot's direction, and:

- Let p be the position of the robot at time t .
- Let p' be the position of the robot at $t + 1$.
- Let c be the position of the turning-point of the robot.
- Let R be the radius from the turning-point c to the inner wheel of the robot.
- Let w be the width of the robot.
- Let l be the distance travelled along the segment of the circle by the left wheel.
- Let r be the distance travelled along the segment of the circle by the right wheel.
- Let $\theta \in [-\pi, \pi)$ be the angle of the robot at p .

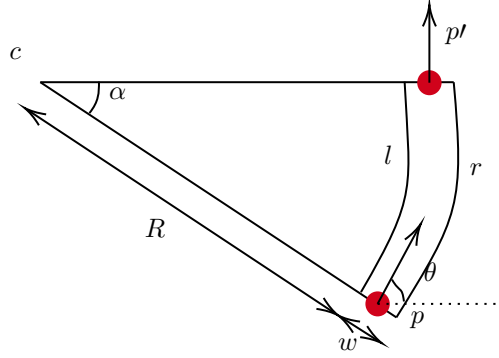


Figure 3.1: Diagram showing robot movement from p to p' .

For $\alpha \neq 0$ we have (from the formula for arc length)

$$l = \alpha R, \quad \text{and} \quad r = \alpha(R + w)$$

hence

$$\alpha = \frac{r - l}{w} \quad \text{and} \quad R = \frac{l}{\alpha}.$$

The direction vector of the robot at p is $\begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix}$ and at p' is $\begin{pmatrix} \cos(\theta + \alpha) \\ \sin(\theta + \alpha) \end{pmatrix}$.

The clockwise orthogonal direction vector at p and p' , is $\begin{pmatrix} \sin \theta \\ -\cos \theta \end{pmatrix}$ and $\begin{pmatrix} \sin(\theta + \alpha) \\ -\cos(\theta + \alpha) \end{pmatrix}$ respectively.

Therefore

$$\begin{aligned} c &= p - \left(R + \frac{w}{2}\right) \begin{pmatrix} \sin \theta \\ -\cos \theta \end{pmatrix}, \\ p' &= c + \left(R + \frac{w}{2}\right) \begin{pmatrix} \sin(\theta + \alpha) \\ -\cos(\theta + \alpha) \end{pmatrix}, \\ \theta' &= (\theta + \alpha) \bmod 2\pi. \end{aligned}$$

Given positions (x, y, θ) at time t , the positions at $t + 1$ are, if $\alpha \neq 0$,

$$\begin{aligned} \begin{pmatrix} x' \\ y' \end{pmatrix} &= \begin{pmatrix} x \\ y \end{pmatrix} + \left(R + \frac{w}{2}\right) \left[\begin{pmatrix} \sin(\theta + \alpha) \\ -\cos(\theta + \alpha) \end{pmatrix} - \begin{pmatrix} \sin \theta \\ -\cos \theta \end{pmatrix} \right], \\ \theta' &= (\theta + \alpha) \bmod 2\pi, \end{aligned}$$

and otherwise for $l = r$ (equivalently $\alpha = 0$)

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} + l \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix},$$

$$\theta' = \theta.$$

Define a function g where for $\mathbf{x} = (x, y, \theta)^\top$ and $u = (l, r)$, we have

$$g(\mathbf{x}, u) = g(x, y, \theta, l, r) := \begin{pmatrix} x' \\ y' \\ \theta' \end{pmatrix}. \quad (1)$$

For each time-step, l and r should be estimated. From this, we can use g to get a new estimate for the position of the robot.

4 Initial Setup

At the start, we know no landmarks.

We define our map at the start as $\mathbf{x} = (x, y, \theta)^\top$. That is to say, our map contains the position of the robot and the angle of the robot. As we detect landmarks, we want to add the positions of the landmarks to our map $\mathbf{x}$. We don't know the exact position of the landmarks but we can estimate them.

In EKF, the map $\mathbf{x}$ is modelled by a Gaussian variable with mean $\boldsymbol{\mu}$ and the covariance matrix $\boldsymbol{\Sigma}$. We define $x, y \in \mathbb{R}$ and $\theta \in [-\pi, \pi)$ as the initial robot position so actually we know these with absolute certainty. We can choose $(x, y) = (0, 0)$ and $\theta = 0$, or any other values.

Since we know $\mathbf{x}$ with absolute certainty, our initial $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ are

$$\boldsymbol{\mu} = \begin{pmatrix} x \\ y \\ \theta \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \quad \text{and} \quad \boldsymbol{\Sigma} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

When the robot moves, its position can only be estimated. We therefore don't look at $\mathbf{x}$ any more but only consider $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$. As we discover new landmarks, their exact position remains unknown, but we add and update them to $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$. This will be explained in the following sections.

4.1 Adding a Landmark

For each new landmark, $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ are updated.

Consider a newly discovered landmark with position (x_i, y_i) . Then

$$\boldsymbol{\mu} = \begin{pmatrix} x \\ y \\ \theta \\ \vdots \end{pmatrix} \quad \text{becomes} \quad \begin{pmatrix} x \\ y \\ \theta \\ \vdots \\ x_i \\ y_i \end{pmatrix}$$

and the covariance matrix is extended in the bottom right to become

$$\boldsymbol{\Sigma} \leftarrow \left(\begin{array}{c|cc} \boldsymbol{\Sigma} & & 0 \\ \hline 0 & \sigma_{x_i}^2 & 0 \\ & 0 & \sigma_{y_i}^2 \end{array} \right).$$

When adding a landmark, choose $\sigma_{x_i}^2$ and $\sigma_{y_i}^2$ to be big (e.g. 100 meters), because from one reading we are still uncertain about its position.

As we add landmarks, the size of $\boldsymbol{\mu}$ grows and so does $\boldsymbol{\Sigma}$. With N landmarks added,

$$\boldsymbol{\mu} = (x, y, \theta, x_1, y_1, \dots, x_N, y_N).$$

5 Prediction Step

Suppose that one time-step has passed and that the robot has moved. We get the values of $u = (l, r)$ for how much the robot has moved. In this prediction step we estimate the new position of the robot.

The EKF prediction step is written as the updates

$$\begin{aligned}\boldsymbol{\mu} &\leftarrow g(\boldsymbol{\mu}, u), \\ \boldsymbol{\Sigma} &\leftarrow G\boldsymbol{\Sigma}G^\top + V\Sigma_u V^\top\end{aligned}$$

where g is defined in [Equation 1](#), and where

$$G = \frac{\partial g}{\partial \boldsymbol{\mu}}, \quad V = \frac{\partial g}{\partial u}, \quad \text{and} \quad \Sigma_u = \begin{pmatrix} \sigma_l^2 & 0 \\ 0 & \sigma_r^2 \end{pmatrix}.$$

Here σ_l is standard deviation of the measurement l and σ_r is standard deviation of the measurement r . We let these be constants.

Let's compute the partial derivatives. When the number of landmarks, $N = 0$ the computation simplifies to the following:

For $r \neq l$

$$G = \frac{\partial g}{\partial \boldsymbol{\mu}} = \begin{pmatrix} 1 & 0 & \left(\frac{l}{q} + \frac{w}{2}\right)(\cos(\theta + \alpha) - \cos \theta) \\ 0 & 1 & \left(\frac{q}{\alpha} + \frac{w}{2}\right)(\sin(\theta + \alpha) - \sin \theta) \\ 0 & 0 & 1 \end{pmatrix},$$

and for $r = l$

$$G = \frac{\partial g}{\partial \boldsymbol{\mu}} = \begin{pmatrix} 1 & 0 & -l \sin \theta \\ 0 & 1 & l \cos \theta \\ 0 & 0 & 1 \end{pmatrix}.$$

For V we have

$$V = \frac{\partial g}{\partial u} = \begin{pmatrix} A & C \\ B & D \\ -\frac{1}{w} & \frac{1}{w} \end{pmatrix},$$

where if $r \neq l$

$$\begin{aligned}A &= \frac{wr}{(r-l)^2} \left(\sin \left(\theta + \frac{r-l}{w} \right) - \sin \theta \right) - \frac{r+l}{2(r-l)} \cos \left(\theta + \frac{r-l}{w} \right), \\ B &= \frac{wr}{(r-l)^2} \left(-\cos \left(\theta + \frac{r-l}{w} \right) + \cos \theta \right) - \frac{r+l}{2(r-l)} \sin \left(\theta + \frac{r-l}{w} \right), \\ C &= -\frac{wl}{(r-l)^2} \left(\sin \left(\theta + \frac{r-l}{w} \right) - \sin \theta \right) + \frac{r+l}{2(r-l)} \cos \left(\theta + \frac{r-l}{w} \right), \\ D &= -\frac{wl}{(r-l)^2} \left(-\cos \left(\theta + \frac{r-l}{w} \right) + \cos \theta \right) + \frac{r+l}{2(r-l)} \sin \left(\theta + \frac{r-l}{w} \right).\end{aligned}$$

If $r = l$ then

$$\begin{aligned}A &= \frac{1}{2} \left(\cos \theta + \frac{l}{w} \sin \theta \right), \\ B &= \frac{1}{2} \left(\sin \theta - \frac{l}{w} \cos \theta \right), \\ C &= \frac{1}{2} \left(\cos \theta - \frac{l}{w} \sin \theta \right), \\ D &= \frac{1}{2} \left(\sin \theta + \frac{l}{w} \cos \theta \right).\end{aligned}$$

When the number of landmarks, $N > 0$, we have

$$G \leftarrow \left(\begin{array}{c|c} G & 0 \\ \hline 0 & 1 \end{array} \right), \quad V \leftarrow \left(\begin{array}{c} V \\ \hline 0 \end{array} \right)$$

where G is a $(3 + 2N) \times (3 + 2N)$ matrix, and V is a $(3 + 2N) \times 2$ matrix.

6 Correction Step

Given the robot position and direction (x, y, θ) . Landmark observations are processed in the EKF one-by-one. Consider a landmark with position (x_m, y_m) in world co-ordinates, with distance r and bearing β to the robot.

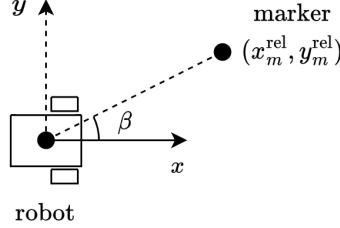


Figure 6.1: Robot Coordinate system, with marker at position $(x_m^{\text{rel}}, y_m^{\text{rel}})$ with angle β . In real world coordinates we denote the marker position as (x_m, y_m) .

Consider the following constants:

Let σ_r be the measurement distance standard deviation.

Let σ_β be the measurement angle standard deviation.

Choose the values of σ_r^2 and σ_β^2 depending on how accurate your measurements are.

We have

$$r = \sqrt{(x_m - x)^2 + (y_m - y)^2},$$

$$\beta = \arctan\left(\frac{y_m - y}{x_m - x}\right) - \theta.$$

Note this is equivalent to

$$r = \sqrt{(x_m^{\text{rel}})^2 + (y_m^{\text{rel}})^2},$$

$$\beta = \arctan\left(\frac{y_m^{\text{rel}}}{x_m^{\text{rel}}}\right).$$

Use the `arctan2` function for computing the arctangent.

Consider the function

$$h(x, y, \theta, x_m, y_m) := \begin{pmatrix} r \\ \beta \end{pmatrix}.$$

That is to say, given the robot position and orientation, and given the landmark position, h returns the distance and angle to the landmark.

Let $\mu = (x, y, \theta, x_m, y_m)$. Then the jacobian is

$$H = \frac{\partial h}{\partial \mu} = \begin{pmatrix} \frac{\partial r}{\partial x} & \frac{\partial r}{\partial y} & \frac{\partial r}{\partial \theta} & \frac{\partial r}{\partial x_m} & \frac{\partial r}{\partial y_m} \\ \frac{\partial \beta}{\partial x} & \frac{\partial \beta}{\partial y} & \frac{\partial \beta}{\partial \theta} & \frac{\partial \beta}{\partial x_m} & \frac{\partial \beta}{\partial y_m} \end{pmatrix},$$

where

$$\begin{aligned}
\frac{\partial r}{\partial x} &= -\frac{x_m - x}{\sqrt{(x_m - x)^2 + (y_m - y)^2}}, & \frac{\partial \beta}{\partial x} &= \frac{y_m - y}{(x_m - x)^2 + (y_m - y)^2}, \\
\frac{\partial r}{\partial y} &= -\frac{y_m - y}{\sqrt{(x_m - x)^2 + (y_m - y)^2}}, & \frac{\partial \beta}{\partial y} &= -\frac{x_m - x}{(x_m - x)^2 + (y_m - y)^2}, \\
\frac{\partial r}{\partial \theta} &= 0, & \frac{\partial \beta}{\partial \theta} &= -1, \\
\frac{\partial r}{\partial x_m} &= \frac{x_m - x}{\sqrt{(x_m - x)^2 + (y_m - y)^2}}, & \frac{\partial \beta}{\partial x_m} &= -\frac{y_m - y}{(x_m - x)^2 + (y_m - y)^2}, \\
\frac{\partial r}{\partial y_m} &= \frac{y_m - y}{\sqrt{(x_m - x)^2 + (y_m - y)^2}}, & \frac{\partial \beta}{\partial y_m} &= \frac{x_m - x}{(x_m - x)^2 + (y_m - y)^2}.
\end{aligned}$$

Notice that

$$\left(\frac{\partial r}{\partial x}, \frac{\partial r}{\partial y}, \frac{\partial \beta}{\partial x}, \frac{\partial \beta}{\partial y} \right) = - \left(\frac{\partial r}{\partial x_m}, \frac{\partial r}{\partial y_m}, \frac{\partial \beta}{\partial x_m}, \frac{\partial \beta}{\partial y_m} \right).$$

For N landmarks we have $\boldsymbol{\mu} = (x, y, \theta, x_1, y_1, \dots, x_N, y_N)$ and

$$\frac{\partial r}{\partial x_i}, \frac{\partial r}{\partial y_i}, \frac{\partial \beta}{\partial x_i}, \frac{\partial \beta}{\partial y_i}$$

for $1 \leq i \leq N$ is equivalent to the case for x_m, y_m above.

Consider a landmark i with $1 \leq i \leq N$ which we want to update. For the next calculations, we write H as follows, where H is a $2 \times (3 + 2N)$ matrix and where the partial derivatives for i are located in the columns $3 + 2i - 1$ and $3 + 2i$,

$$H = \begin{pmatrix} \frac{\partial r}{\partial x} & \frac{\partial r}{\partial y} & \frac{\partial r}{\partial \theta} & 0 & 0 & \dots & \frac{\partial r}{\partial x_i} & \frac{\partial r}{\partial y_i} & \dots & 0 & 0 \\ \frac{\partial \beta}{\partial x} & \frac{\partial \beta}{\partial y} & \frac{\partial \beta}{\partial \theta} & 0 & 0 & \dots & \frac{\partial \beta}{\partial x_i} & \frac{\partial \beta}{\partial y_i} & \dots & 0 & 0 \end{pmatrix}.$$

We have now computed H . Now we will describe the correction step procedure. This procedure should be computed for each landmark that is seen by the robot at this time-step.

Suppose we get a new measurement for landmark i with actual measurement (r_i, β_i) . This procedure is the same as the correction step in the EKF.

Let Q be the covariance matrix representing the uncertainty of (r_i, β_i) .

$$Q := \begin{pmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\beta^2 \end{pmatrix}.$$

Let

$$K := \Sigma \left[H^\top (H \Sigma H^\top + Q)^{-1} \right]. \quad (2)$$

The actual measurement is (r_i, β_i) and our prediction of this measurement is $h(x, y, \theta, x_i, y_i)$, where we get the values x, y, θ, x_i, y_i from $\boldsymbol{\mu}$.

Update $\boldsymbol{\mu}$ with

$$\boldsymbol{\mu} \leftarrow \boldsymbol{\mu} + K \left((r_i, \beta_i)^\top - h(x, y, \theta, x_i, y_i) \right).$$

Update Σ with

$$\Sigma \leftarrow (I - KH) \Sigma,$$

where I is the identity matrix with size $(3 + 2N) \times (3 + 2N)$.

Note: be careful taking the difference of angles. We should have $(r_i, \beta_i)^\top - h(x, y, \theta, x_i, y_i) \in [-\pi, \pi)$.

Repeat this procedure for each landmark seen by the camera at this time-step.

6.1 Reducing Computational Complexity

Let $Z = (H\Sigma H^\top + Q)^{-1}$ in Equation 2.

We can reduce the computational complexity of computing the inverse in Z .

Let Σ_s be the 5×5 matrix given by the 3×3 submatrix corresponding to x, y, θ , and the 2×2 submatrix corresponding to x_i, y_i . Include the cross-covariances of these two submatrices. Namely

$$\Sigma_s = \left(\begin{array}{c|c} \Sigma_{1:3,1:3} & \text{cross-covariances} \\ \hline \text{cross-covariances} & \Sigma_{(3+2i-1):(3+2i), (3+2i-1):(3+2i)} \end{array} \right).$$

Let

$$H_s = \begin{pmatrix} \frac{\partial r}{\partial x} & \frac{\partial r}{\partial y} & \frac{\partial r}{\partial \theta} & \frac{\partial r}{\partial x_i} & \frac{\partial r}{\partial y_i} \\ \frac{\partial \beta}{\partial x} & \frac{\partial \beta}{\partial y} & \frac{\partial \beta}{\partial \theta} & \frac{\partial \beta}{\partial x_i} & \frac{\partial \beta}{\partial y_i} \end{pmatrix}.$$

Then we can see that, $Z = (H_s \Sigma_s H_s^\top + Q)^{-1}$.

7 The Current Map

At each time-step, the map of the estimated robot position and orientation, and the position of all the landmarks is stored in μ with covariance Σ .

We want to represent the error in the position of the landmark or robot as an ellipse. For a pair (x_i, y_i) in μ , which could be the robot position or a landmark position, we have a corresponding 2×2 matrix in Σ . Let's call it Σ_i .

Write Σ_i as

$$\Sigma_i = \begin{pmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{pmatrix}.$$

When x and y are independent $\sigma_{xy} = 0$. When x and y are correlated, then $\sigma_{xy} \neq 0$.

With eigendecomposition we have

$$\Sigma_i = V \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix} V^\top,$$

where

$$V = \begin{pmatrix} | & | \\ v_1 & v_2 \\ | & | \end{pmatrix} \quad \text{eigenvectors, and } \lambda_1, \lambda_2 \text{ eigenvalues.}$$

The eigenvalues give the length of the eigenvectors. Let $\lambda_1 \geq \lambda_2$. The eigenvectors v_1, v_2 correspond to λ_1, λ_2 respectively. Write $v_1 = (v_{11}, v_{12})^\top$. Then the angle of v_1 with the horizontal is $\gamma = \arctan\left(\frac{v_{12}}{v_{11}}\right)$.

We can draw this ellipse given the position μ_x, μ_y and $\sqrt{\lambda_1}, \sqrt{\lambda_2}$ as the two size components of the ellipse. We take the square-root because we want the ellipse size to represent the standard deviation. The ellipse is at angle γ .

In the robot viewer application, we provide the parameters in the order: $\sqrt{\lambda_1}, \sqrt{\lambda_2}, \gamma$.

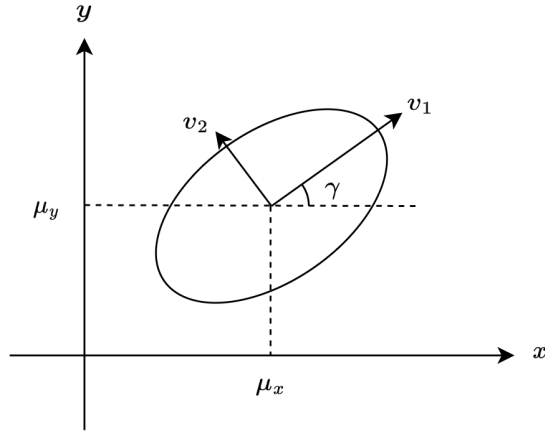


Figure 7.1: Standard deviation shown as an ellipse.

8 Computing the difference of two angles

Let $\theta_1, \theta_2 \in [-\pi, \pi)$. We compute the difference in the two angles as follows

$$\theta_{1-2} = \begin{cases} ((\theta_1 - \theta_2) \bmod 2\pi) - 2\pi & \text{if } (\theta_1 - \theta_2) \bmod 2\pi > \pi, \\ (\theta_1 - \theta_2) \bmod 2\pi & \text{else.} \end{cases}$$